



Installing the ggoss26 conda environment

This guide installs **the Python/conda environment** defined in `ggosss2026_conda_env.yaml`. That environment is the one used by

- **croco_work**: the CROCO pre-processing tools (`gtools/` , `croco_pytools`), downloads (CMEMS, ERA5), plotting and validation; and
- the **OpenDrift exercises** (`open_drift/*.ipynb`).

Done once per machine. After the install, every session only needs `conda activate ggoss26`.

0. Pick your platform

Platform	What you use
Linux (Ubuntu 20.04+, etc.)	A normal terminal.
macOS 12+ (Intel or Apple Silicon)	The Terminal app. Every package in the YAML has macOS builds (<code>osx-64</code> / <code>osx-arm64</code>) on conda-forge.
Windows 10/11	WSL2 + Ubuntu . CROCO and its tools are built for Linux, so don't use a native Windows conda.

Windows only: install WSL2

1. Open **PowerShell as Administrator** and run:

```
ws1 --install -d Ubuntu
```

You can also install **Ubuntu** from the Microsoft Store.

2. Restart if asked. Open **Ubuntu** from the Start menu and create a UNIX username and password. The password isn't shown while you type, which is expected.
3. Check that you're in Linux:

```
uname -a      # should mention "microsoft-standard-WSL2"
```

From here on, **every command runs in the Ubuntu terminal**.

Put your files in the Linux home directory (`~/`), not under `/mnt/c/...`. Working from the Windows drive is many times slower. To copy the YAML over from Windows downloads

(replace `<WinUser>` with your Windows user name):

In [1]:

```
cp /mnt/c/Users/<WinUser>/Downloads/ggoss2026_conda_env.yaml ~/
```

bash

To see the Linux files from Windows Explorer, go to `\\wsl$\\Ubuntu\\home\\<linux-user>` in the Windows Explorer address bar. If your `linux-user` is for example `student`, then `\\wsl$\\Ubuntu\\home\\student`.

1. Install conda (Miniconda)

Install Miniconda into the default location, `~/miniconda3`.

Download the installer for your machine:

Machine	Installer
Linux / WSL2, x86_64	<code>Miniconda3-latest-Linux-x86_64.sh</code>
Linux, ARM (aarch64)	<code>Miniconda3-latest-Linux-aarch64.sh</code>
macOS, Apple Silicon (M1–M4)	<code>Miniconda3-latest-MacOSX-arm64.sh</code>
macOS, Intel	<code>Miniconda3-latest-MacOSX-x86_64.sh</code>

In [2]:

```
cd ~  
# Linux / WSL2 (x86_64). On macOS, replace the file name using the table above;  
# curl is used because macOS has no wget by default.  
curl -LO https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh  
bash Miniconda3-latest-Linux-x86_64.sh
```

bash

- Accept the licence.
- Keep the default location (`~/miniconda3`).
- Answer **yes** when asked whether to initialise conda.

Close and reopen the terminal, or run `source ~/.bashrc` (on macOS, `source ~/.zshrc`). The prompt should now start with `(base)`.

In [3]:

```
conda --version
```

bash

Use conda-forge only (recommended)

The YAML only uses `conda-forge`. Taking Anaconda's `defaults` channel out of the configuration prevents mixed-channel conflicts. It also avoids the “Terms of Service”

prompt that recent Miniconda versions show for `defaults`:

```
In [4]: conda config --add channels conda-forge
conda config --remove channels defaults      # ignore "not found" if it isn't
there
conda config --set channel_priority strict
```

bash

If conda still asks you to accept the Terms of Service for `repo.anaconda.com` channels, either accept them (`conda tos accept`) or run the `--remove channels defaults` line again.

2. Create the environment

Go to the folder that holds the YAML (`~` if you copied it there):

```
In [5]: cd ~
conda env create -f ggoss2026_conda_env.yaml
```

bash

- The environment name comes from the first line of the file, `name: ggoss26`. The `croco_work` scripts activate exactly that name, so don't rename it.
- Creating the environment takes about **10–30 minutes**: it solves roughly 50 conda packages, then pip installs `regionmask`, `pydap` and `opendrift` (OpenDrift pulls in `roaring-landmask`, `adios_db` and more).
- Don't interrupt it. If it fails part-way, see [Troubleshooting](#).

Activate it:

```
In [6]: conda activate ggoss26
```

bash

The prompt now shows `(ggoss26)`.

3. Verify the environment

3a. Imports (croco_work + OpenDrift)

In [7]:

```
conda activate ggosss26
python - <<'EOF'
import xarray, netCDF4, numpy, scipy, pandas, dask, h5netcdf, zarr, cftime
import cartopy, geopandas, shapely, pyproj, pyinterp, regionmask
import copernicusmarine, cdsapi, cf_grib, pydap
import xgcm, xrft, cf_xarray, numba, pyang
import opendrift, cmocean, gsw, sklearn
from opendrift.models.oceandrift import OceanDrift
from opendrift.models.leeway import Leeway
from opendrift.models.openoil import OpenOil
from opendrift.readers import reader_netCDF_CF_generic, reader_ROMS_native
print("numpy", numpy.__version__, "| xarray", xarray.__version__, "| opendrift",
opendrift.__version__)
print("ggosss26 env OK")
EOF
```

bash

You should see `ggosss26 env OK`.

3b. OpenDrift smoke test (no data needed)

In [8]:

```
python - <<'EOF'
from datetime import datetime, timedelta
from opendrift.models.oceandrift import OceanDrift
o = OceanDrift(loglevel=50)
o.set_config('environment:fallback:x_sea_water_velocity', 0.2)
o.set_config('environment:fallback:y_sea_water_velocity', 0.1)
o.seed_elements(lon=3.0, lat=4.0, number=10, time=datetime(2026, 9, 18))
o.run(duration=timedelta(hours=6), time_step=3600)
print("OpenDrift OK:", o.elements.lon.mean().round(3),
o.elements.lat.mean().round(3))
EOF
```

bash

The first run downloads nothing. The `roaring-landmask` coastline ships with the package.

4. Troubleshooting

Symptom	Fix
<code>conda: command not found</code>	Reopen the terminal, or run <code>source ~/miniconda3/etc/profile.d/conda.sh</code> .

Symptom	Fix
YAML parse error on <code>conda env create</code>	The first line must be exactly <code>name: ggosss26</code> , with a space after the colon. Don't edit the file in Word or Notepad with "smart" formatting.
Terms of Service error for <code>repo.anaconda.com</code>	See "Use conda-forge only" in step 1, or run <code>conda tos accept</code> .
Conda part OK, pip part failed (network or proxy)	Finish it by hand: <code>conda activate ggosss26 && pip install regionmask pydap opendrift</code>
Solve takes forever or conflicts	Update conda (<code>conda update -n base conda</code>) and make sure <code>channel_priority</code> is <code>strict</code> . Then start over: <code>conda env remove -n ggosss26</code> and create it again.
Environment exists but is broken or old	<code>conda env remove -n ggosss26</code> , then <code>conda env create -f ggosss2026_conda_env.yaml</code>
WSL2: install very slow, or files "missing"	Work in <code>~</code> (Linux filesystem), not in <code>/mnt/c/...</code> .
WSL2: runs out of memory	Create <code>C:\Users\<WinUser>\.wslconfig</code> with <code>[wsl2]</code> and <code>memory=8GB</code> on the next line. Then run <code>wsl --shutdown</code> in PowerShell.
<code>import wx</code> fails on a server with no display	Only <code>croco_pyvisu</code> 's GUI needs wxPython. The rest of the environment is unaffected.

Summary

Example for Linux / Windows (WSL2):

In [9]:

```
# once per machine
cd ~
curl -LO https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh          # -> ~/miniconda3
source ~/.bashrc                               # or reopen the terminal
conda config --add channels conda-forge && conda config --set channel_priority
strict

cp /mnt/c/Users/<WinUser>/Downloads/ggosss2026_conda_env.yaml ~/
conda env create -f ggosss2026_conda_env.yaml    # -> env "ggosss26"
conda activate ggosss26
python -m ipykernel install --user --name ggosss26 --display-name "Python
(ggosss26)"

# every session
conda activate ggosss26
```

bash